1. 多播和广播

1.1 多播

1.1.1 定义

多播（Multicast）也称为组播，是一种一对多的通信方式，将信息从单个源发送到多个特定的接收者。这些接收者组成一个特定的多播组，只有加入该组的设备才会接收和处理多播数据。

1.1.2 特点

- **高效性**：多播避免了向每个接收者单独发送数据的重复传输，从而节省了网络带宽和发送方的资源。例如，在视频会议应用中，一个演讲者的视频和音频流可以通过多播发送给多个参与者，而不是为每个参与者单独发送一份相同的流。
- **可管理性**：多播组可以动态地加入和退出，便于管理和控制接收者的范围。例如，在网络游戏中，玩家可以根据自己的游戏进程加入或退出特定的多播组，以接收与特定游戏场景相关的信息。
- **协议支持**：多播通常使用特定的协议，如 IP 多播（使用 D 类 IP 地址范围 224.0.0.0 到 239.255.255.255）。

1.1.3 应用场景

- **多媒体流传输**：在线视频和音频流服务可以使用多播将内容同时发送给多个用户，减少服务器的负载和网络带宽的占用。例如，网络电视、在线教育平台等可以利用多播技术向大量用户同时推送视频课程或直播节目。
- **企业内部通信**：企业内部的视频会议、数据分发等应用可以使用多播来提高效率。例如，公司内部的培训课程可以通过多播发送给分布在不同地点的员工，而不需要为每个员工单独建立连接。
- **网络游戏**：多人在线游戏可以使用多播来同步游戏状态和事件。例如，游戏中的怪物移动、玩家交互等信息可以通过多播发送给同一游戏区域内的玩家，减少服务器的通信负担。

1.2 广播

华清远见嵌入式实验中心 www.yvzlab.com.cn

1.2.1 定义

广播（Broadcast）是一种一对所有的通信方式，将信息从单个源发送到网络中的所有设备。广播数据包在网络中被广泛传播，任何连接到该网络的设备都可以接收到广播消息。

1.2.2 特点

- **广泛性**：广播消息可以覆盖整个网络，确保所有设备都有机会接收到信息。这在某些情况下非常有用，例如网络发现、设备初始化等。
- **简单性**：广播通信相对简单，不需要复杂的路由和组管理机制。发送方只需将数据包发送到广播地址，网络中的设备会自动接收并处理广播消息。
- **安全性**：广播通信的安全性较低，因为任何连接到网络的设备都可以接收到广播消息。因此当网络中存在大量的广播数据包进行传输时，会导致网络的性能严重下降。

1.2.3 应用场景

- **网络发现**：设备可以通过发送广播消息来发现网络中的其他设备和服务。例如，计算机在启动时可以发送广播请求，以查找可用的打印机、文件服务器等资源。
- **DHCP 服务**：动态主机配置协议（DHCP）使用广播来分配 IP 地址和其他网络配置参数。当计算机启动时，它会发送一个 DHCP 广播请求，网络中的 DHCP 服务器会响应这个请求，为计算机分配一个 IP 地址和其他必要的配置信息。
- **网络管理**：网络管理员可以使用广播来发送管理命令或查询网络状态，如检查网络中的设备是否在线、收集网络统计信息等。

2. TCP

传输控制协议（TCP，Transmission Control Protocol）是一种面向连接的、可靠的、基于字节流的传输层通信协议。

2.1 协议格式

华清远见嵌入式实验中心 yyzlab.com.cn

源端口和目的端口：各占16位，用于标识发送方和接收方的应用程序进程。

序号：32位，标识发送数据包中的第一个字节的序列号。

确认序号：32位，是期望收到的下一个数据包的序列号，用于确认已经收到的数据包。

头部长度：4位，TCP首部的长度。

保留：6位，保留位，暂时没有什么作用。

标志位：6位，包括 URG（紧急指针有效）、ACK（确认号有效）、PSH（推送功能）、RST（复位 TCP 连接）、SYN（同步报文，用于建立连接）、FIN（没有数据需要发送，用于关闭连接）。

窗口：16位，用来告诉TCP连接对端自己能够接收的最大数据长度。

校验和：16位，用来检验数据的完整性。

紧急指针：16位，只有 URG 标志位被设置时该字段才有意义，表示紧急数据相对序列号的偏移。

2.2 特点

- **面向连接**：在通信之前，需要通过特定的握手过程在发送方和接收方之间建立连接。连接建立后，数据可以在这个连接上可靠地传输，通信结束后还需要通过特定的挥手过程来释放连接。
- **可靠性高**：
  - **序号与确认机制**：为每个发送的数据包分配一个序号，接收方根据序号对收到的数据包进行排序和确认。发送方如果在合理的往返时延内没有收到确认，就会重传数据包。
  - **校验和**：计算数据包的校验和，用于检测数据在传输过程中是否发生错误。如果收到的数据包校验和有差错，TCP 将丢弃这个分片。

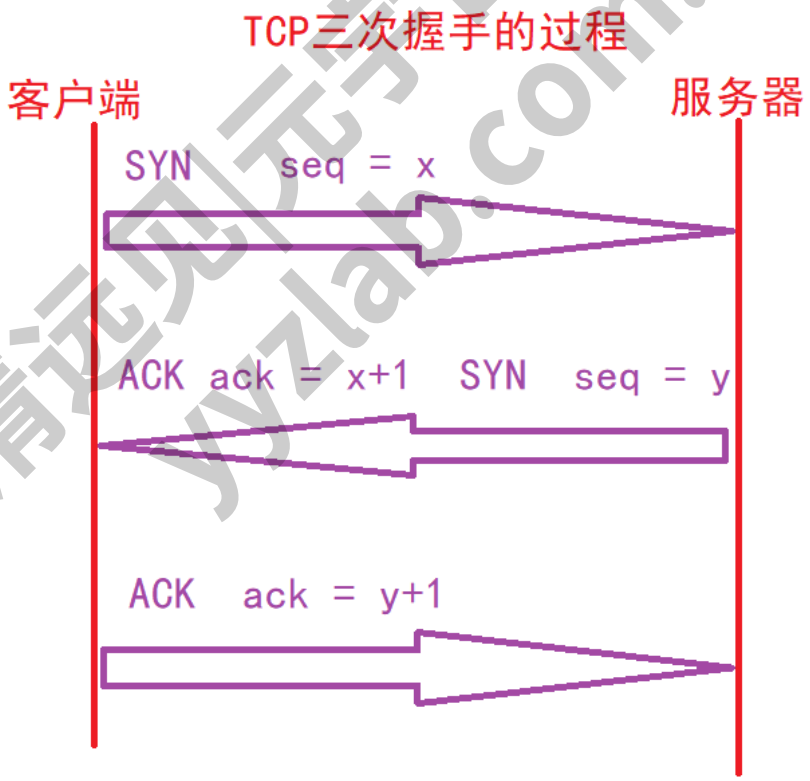
华清远见嵌入式实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- **流量控制**：通过滑动窗口协议实现。接收方在确认数据包时会告知发送方自己的接收窗口大小，发送方根据这个窗口大小来控制发送数据的速率，防止发送过快导致接收方缓冲区溢出。
- **拥塞控制**：能够根据网络的拥塞情况动态调整发送数据的速率，避免网络拥塞。例如采用慢启动、拥塞避免、快速重传和快速恢复等算法。

2.3 工作方式

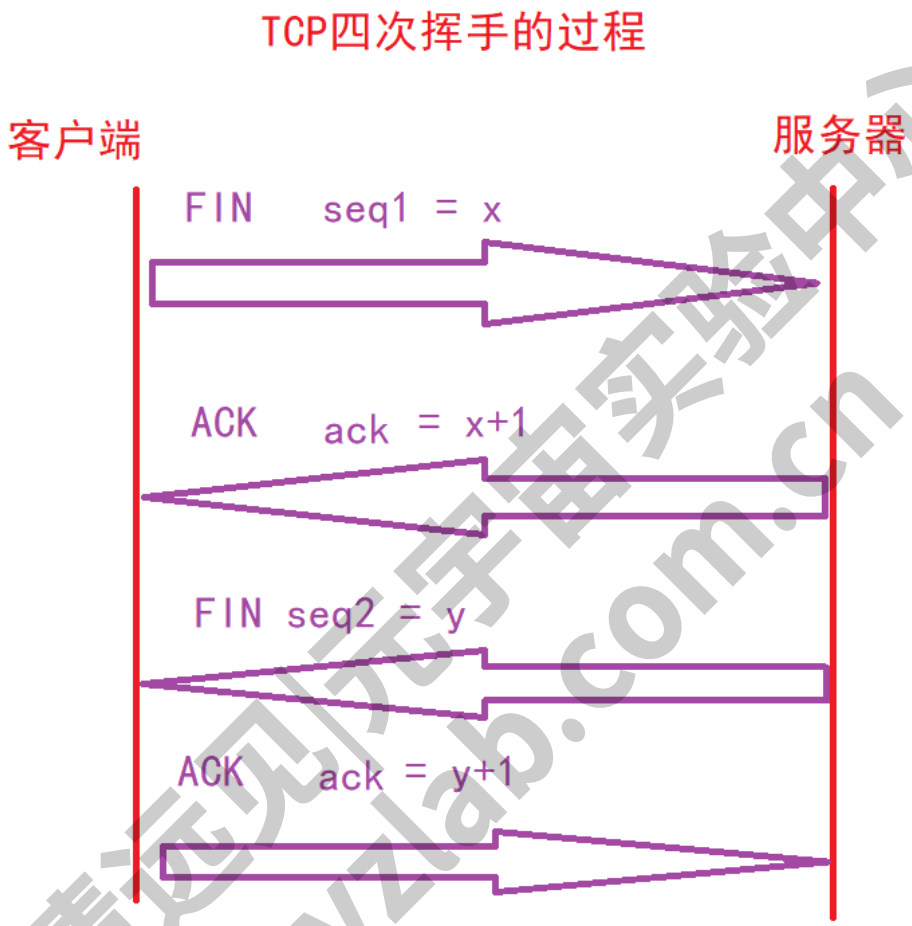
- **建立连接（三次握手）**：
  - 客户端发送 SYN（同步）报文给服务器端，进入 SYN_SEND 状态，请求建立连接，其中包含一个随机生成的初始序列号。
  - 服务器端收到 SYN 报文后，回应一个 SYN（seq = y）ACK（ack = x + 1）报文，进入 SYN_RECV 状态，表示收到了客户端的请求，并向客户端确认。
  - 客户端收到服务器端的 SYN 报文后，回应一个 ACK（ack = y + 1）报文，进入 ESTABLISHED 状态。服务器端接收到这个 ACK 报文后，也进入 ESTABLISHED 状态，此时连接建立成功，可以开始传输数据。



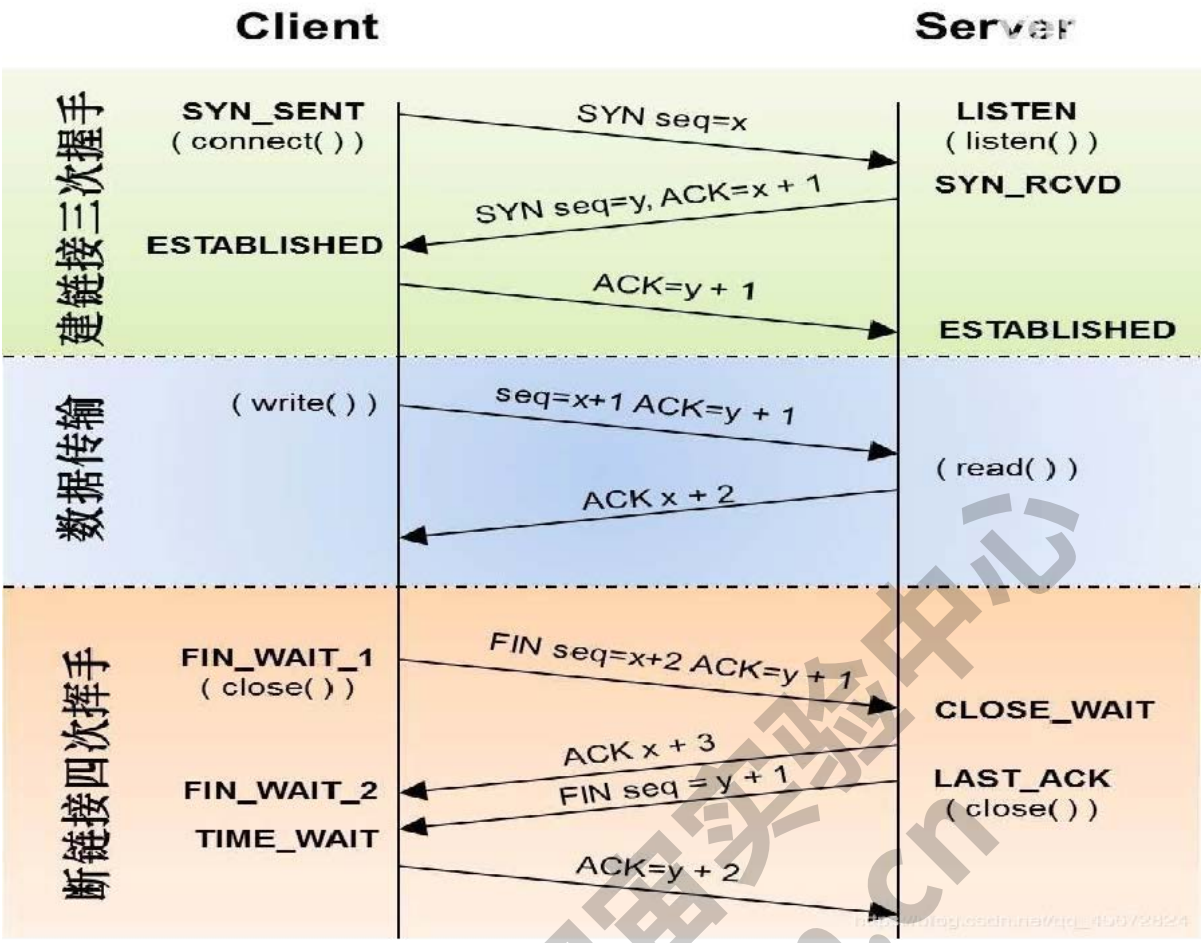
- **数据传输**：发送方将应用层的数据分割成适当大小的报文段，加上 TCP 首部后发送给接收方。接收方收到报文段后，进行确认，并将数据交给应用层。
- 断开连接（四次挥手）：
  - 客户端发送一个 FIN（结束）报文，表明自己没有数据要发送了，进入 FIN_WAIT_1 状态。

华清远见云实验中心 yyzlab.com.cn

- 服务器端收到 FIN 报文后，发送一个 ACK 报文，确认客户端的 FIN 请求，进入 CLOSE_WAIT 状态。客户端收到 ACK 报文后，进入 FIN_WAIT_2 状态。
- 服务器端处理完剩余的数据后，也发送一个 FIN 报文，进入 LAST_ACK 状态。
- 客户端收到服务器端的 FIN 报文后，发送一个 ACK 报文，确认服务器端的 FIN 请求，然后进入 TIME_WAIT 状态。经过一段时间的等待后，连接彻底关闭。



华清远见宇宙实验中心 yyzlab.com.cn



2.4 应用场景

- **文件传输**：如 FTP（文件传输协议），需要保证文件的完整传输，使用 TCP 协议可以确保文件传输的准确性和可靠性。
- **电子邮件**：SMTP（简单邮件传输协议）等电子邮件协议使用 TCP 协议来保证邮件的可靠传输，避免邮件丢失或损坏。
- **Web 浏览**：HTTP（超文本传输协议）基于 TCP 协议，确保网页内容的正确传输和显示。

3. UDP

用户数据报协议（UDP，User Datagram Protocol）是 OSI（开放系统互连）参考模型中一种无连接的传输层协议，提供面向事务的简单不可靠信息传送服务。

3.1 协议格式

华清远见 HQYJ.COM
华清远见宇宙实验中心 yyzlab.com.cn

0	1	2	3
源端口		目的端口	
UDP长度		校验和	

源端口：16位，标识发送端的端口号，用于区分不同的应用程序或进程。

目的端口：16位，标识接收端的端口号，使数据报能够被正确地路由到接收端的应用程序。

UDP长度：16位，表示 UDP 数据报的长度，包括 UDP 首部和数据部分的长度。

校验和：16位，用于检测数据在传输过程中是否发生了错误。发送端计算数据报的校验和并将其添加到 UDP 首部中，接收端根据接收到的数据报重新计算校验和，并与数据报中的校验和进行比较，以检测数据的完整性。如果校验和不匹配，说明数据报可能在传输过程中被篡改或损坏，但 UDP 本身不提供错误修正机制。

3.2 特点

无连接性：通信双方在传输数据之前不需要建立连接，也不需要维护连接状态。发送方可以直接向接收方发送数据报，无需事先进行握手等操作，减少了连接建立和维护的时间开销。

不可靠性：

- 不提供数据包的确认和重传机制，数据传输过程中可能会出现丢包、重复或乱序等情况。如果数据包在传输过程中丢失或损坏，UDP 协议本身不会采取任何措施来恢复数据。
- 没有流量控制和拥塞控制机制，不能根据网络的拥塞情况调整发送数据的速率，可能会导致网络拥塞加剧。

面向数据报：以数据报为基本单位进行通信，每个数据报是一个独立的、完整的消息，具有独立的头部和数据部分。UDP 不会对数据进行拆分或合并，保持数据报的边界。

轻量级：UDP 协议的头部开销较小，只有 8 个字节，相比 TCP 协议的 20 个字节头部，UDP 可以减少网络传输中的额外开销。

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

快速性：由于不需要建立连接和维护状态，以及头部开销小，UDP 传输速度较快，适用于对实时性要求较高的应用场景。

3.3 应用场景

- **实时多媒体传输**：如实时视频会议、网络电话（VoIP）、在线游戏等实时性要求高的应用。在这些应用中，偶尔丢失一两个数据包对接收结果的影响较小，相比之下，更注重数据的实时传输，UDP 的低延迟和快速性能够满足要求。
- **广播和多播通信**：UDP 不需要与每个接收者建立连接，能够更有效地向多个目标同时发送数据，适用于广播和多播通信模式，如视频直播、网络电台等应用。
- **简单的网络应用**：对于一些对数据可靠性要求不高的简单网络应用，如域名系统（DNS）查询。即使偶尔出现数据包丢失，也可以通过重新查询来获取正确的结果。
- **快速数据传输**：在数据传输时间很短，数据量较小，且对数据完整性要求不高的情况下，UDP 是一个合适的选择，例如一些传感器数据的采集和传输。

4. socket

socket（套接字）是网络通信中的一个抽象概念，它是计算机之间进行通信的端点。在 Python 中，使用 `socket` 模块可以创建不同类型的套接字，实现客户端与服务器之间的通信。

4.1 socket的创建

```
socket.socket(family=AF_INET, type=SOCK_STREAM, proto=0,
fileno=None)
```

family：指定地址族，决定了套接字可以使用的地址类型。

- **socket.AF_INET**：这是最常见的地址族，表示IPV4地址。使用这个地址族创建的套接字可以在互联网上与其他IPV4主机进行通信。
- **socket.AF_INET6**：表示IPV6地址。
- **socket.AF_UNIX**：用于在同一台计算机上的不同进程之间进行通信。这种通信方式不涉及网络，而是通过文件系统中的特殊文件实现。
- **socket.AF_CAN**：用于Linux内核中实现CAN总线通信。CAN总线是一种广泛应用于汽车、工业自动化等领域的现场总线，是一种通信协议。
- **socket.AF_PACKET**：允许直接访问数据链路层的数据包，这意味着可以在不经过网络协议栈的高层处理的情况下，捕获和发送原始的网络数据包。

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- **socket.AF_RDS**：提供一种可靠的数据包通信机制，类似于UDP但具有更高的可靠性。

type：套接字类型。

- **socket.SOCK_STREAM**：面向连接的流式套接字，提供可靠的、顺序的、双向的字节流通信。这通常用于像 TCP（Transmission Control Protocol）这样的协议，确保数据的完整性和顺序性。例如，在进行文件传输或网页浏览时，通常使用这种类型的套接字。
- **socket.SOCK_DGRAM**：无连接的数据报套接字，提供不可靠的、不保证顺序的通信。这种类型的套接字通常用于像 UDP（User Datagram Protocol）这样的协议，适用于对实时性要求较高但对数据可靠性要求不高的应用，如实时视频流或在线游戏。
- **socket.RAW**：原始套接字，可以直接访问网络层的数据，绕过了传输层（如 TCP 和 UDP）的封装。这意味着你可以自己构建 IP 数据包、ICMP 数据包等，或者捕获和分析原始的网络数据包。

proto：协议编号，通常为0，表示自动选择合适的协议。

fileno：如果指定了文件描述符，则使用该文件描述符创建套接字，而不是创建新的套接字，通常不指定。

4.2 常用的方法

4.2.1 bind(address)

- 将套接字绑定到一个特定的地址和端口上。这个地址通常是一个元组，包含 IP 地址和端口号。例如，对于 IPv4 地址，`address`可以是`('127.0.0.1', 8888)`，表示将套接字绑定到本地回环地址 127.0.0.1 的 8888 端口上。
- 服务器端通常需要调用这个方法来指定监听的地址和端口，以便客户端能够连接到服务器。

4.2.2 listen(backlog)

- 开始监听传入的连接请求。`backlog`参数指定在拒绝连接之前，可以挂起的最大连接数量。这个参数通常用于控制服务器能够同时处理的连接请求数量。
- 服务器端在绑定地址后，调用这个方法来开始接受客户端的连接请求。

华清远见|元宇宙实验中心 yyzlab.com.cn

4.2.3 accept()

- 接受一个传入的连接。这个方法会阻塞直到有连接请求到来，然后返回一个新的套接字对象和客户端的地址。新的套接字对象可以用于与客户端进行通信，而客户端的地址通常是一个元组，包含客户端的 IP 地址和端口号。
- 服务器端使用这个方法来接受客户端的连接，并创建一个新的套接字与客户端进行通信。

4.2.4 connect(address)

- 连接到指定的地址。`address`是服务器的地址和端口号组成的元组。
- 客户端使用这个方法来连接到服务器。

4.2.5 send(data)

- 发送数据。`data`是要发送的字节数据，这个方法会将数据发送到连接的另一端。必须是字节串类型，如果发送字符串需要先进行encode编码。
- 返回值是实际发送的字节数。如果发送失败，可能会抛出异常。
- TCP通信使用。

4.2.6 recv(bufsize)

- 接收数据。`bufsize`指定一次最多接收的字节数。这个方法会从连接的另一端接收数据，并返回接收到的字节数据。接收到的是字节串，需要使用decode进行解码。
- 如果没有数据可接收，这个方法会阻塞，直到有数据到来或者连接关闭。如果连接关闭，返回一个空字节串。
- TCP通信使用。

4.2.7 close()

- 关闭套接字。这个方法会释放套接字占用的资源，并关闭连接。
- 在通信结束后，无论是服务器端还是客户端都应该调用这个方法来关闭套接字。

4.2.8 sendto(data, address)

- **data**是要发送的数据。必须是字节码，发送时需要先使用encode进行编码。
- **address**：一个二元组，包含目标地址和端口号。例如`('127.0.0.1', 8888)`表示将数据发送到本地回环地址的 8888 端口。

华清远见元宇宙实验中心 yyzlab.com.cn

- 主要用于 UDP 。

4.2.9 recvfrom(bufsize, flags)

- **bufsize**表示要接受的最大字节数。flags是一个可选参数，和系统有关，使用默认值即可。
- 它会返回一个二元组`(data, address)`，其中`data`是接收到的字节串数据，需要使用decode进行解码，`address`是发送方的地址信息，通常是一个二元组`(IP地址, 端口号)`。
- 主要用于 UDP 。
- **与 TCP 的`recv()`对比**：在 TCP 中，数据的接收是通过已经建立的连接进行的，通常使用`recv()`方法，该方法只返回接收到的数据，而不需要同时获取发送方的地址信息，因为在 TCP 连接中已经知道了通信的对端地址。

4.2.10 其他常用函数

1. `getsockname()`：返回套接字自己的地址。
2. `getpeername()`：返回连接对象的地址。
3. `gethostname()`：返回一个字符串，包含当前正在运行 Python 解释器的机器的主机名。
4. `gethostbyname(hostname)`：解析主机名，返回第一个匹配的IP地址。
5. `gethostbyname_ex(hostname)`：解析主机名，返回一个包含主机别名和地址列表的元组。
6. `gethostbyaddr(ip_address)`：返回一个三元组（hostname, aliaslist, ipaddrlist），hostname是主机名，aliaslist是其他可用主机名列表（可能为空），ipaddrlist是ipv4/v6地址列表。
7. `setblocking(bool)`：设置套接字为阻塞或非阻塞模式。
8. `settimeout(value)`：为阻塞套接字的操作设置超时时间，是一个非负数。如果是非零值，那么在操作完成前超过了指定时间，那么就会抛出异常，如果是0，将处于非阻塞模式，如果为None，将处于阻塞模式。

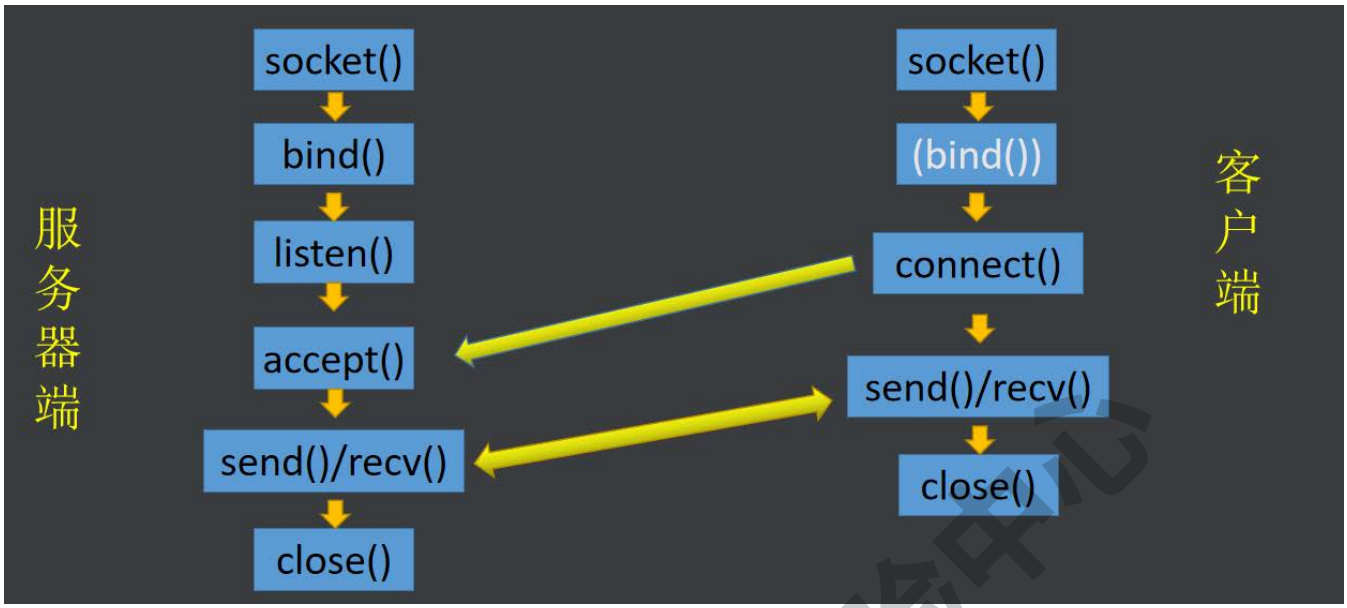
4.3 C/S架构

client：客户端

Server：服务器

华清远见元宇宙实验中心 yyzlab.com.cn

4.3.1 TCP服务器



S：Server，服务端。

```
import socket

# 创建一个 IPv4 的 TCP 套接字
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 定义服务器的地址和端口
server_address = ('127.0.0.1', 43999)

# 将套接字绑定到指定的地址和端口
server_socket.bind(server_address)

# 开始监听连接请求，参数 5 表示允许排队等待的连接数量
server_socket.listen(5)

# # 设置套接字为非阻塞模式
# server_socket.setblocking(False)

# # 为阻塞模式设置超时时间
# server_socket.settimeout(3)


print('服务器正在监听...')

# 接受一个客户端的连接请求，返回一个新的连接套接字和客户端地址
connection, client_address = server_socket.accept()
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
print(f'接受来自{client_address}的连接')

# 从连接套接字接收最多 1024 字节的数据
data = connection.recv(1024)
# 打印接收到的来自客户端的数据
print(f'接收来自{client_address}的数据：{data.decode()}')
# 定义服务器要发送给客户端的响应内容
response = '我是服务器'
# 将响应内容编码后发送给客户端
connection.send(response.encode())

print(f'getsockname: {server_socket.getsockname()}')
print(f'getpeername: {connection.getpeername()}')
print(f'gethostname: {socket.gethostname()}')
print(f'gethostbyname:
{socket.gethostbyname(socket.gethostname())}')
print(f'gethostbyname_ex:
{socket.gethostbyname_ex(socket.gethostname())}')
print(f'gethostbyaddr:
{socket.gethostbyaddr(socket.gethostname())}')


# # 关闭与客户端的连接套接字
# connection.close()
# 关闭服务器的监听套接字
server_socket.close()
```

C：Client，客户端

```
import socket

# 创建一个 IPv4 的 TCP 套接字
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 定义服务器的地址和端口
server_address = ('127.0.0.1', 43999)

# 客户端连接到服务器的地址和端口
client_socket.connect(server_address)

# 客户端向服务器发送经过编码的消息“我是客户端”
```

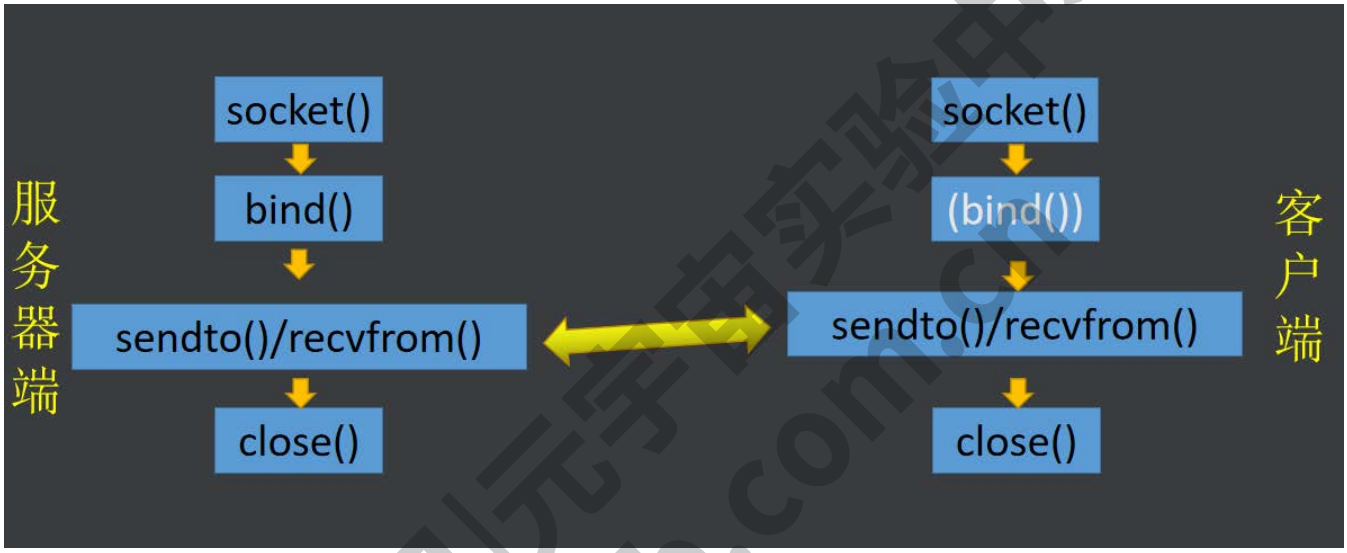
华清远见|元宇宙实验中心 yyzlab.com.cn

```
client_socket.send('我是客户端'.encode())

# 客户端接收来自服务器的最多 1024 字节的数据
data = client_socket.recv(1024)
# 打印接收到的来自服务器的响应消息
print(f'来自服务器的响应为：{data.decode()}')

# 关闭客户端套接字
client_socket.close()
```

4.3.2 UDP服务器



Server：

```
import socket

# 创建一个 IPv4 的 UDP 套接字
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# 定义服务器的地址和端口
server_address = ('127.0.0.1', 43999)

# 将套接字绑定到指定的地址和端口
server_socket.bind(server_address)

print('服务器正在监听...')


# 接收客户端发送的数据以及客户端的地址
data, client_address = server_socket.recvfrom(1024)
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
print(f'接收来自{client_address}的数据：{data.decode()}')
# 定义服务器要发送给客户端的响应内容
response = '我是服务器'
# 将响应内容发送给客户端
server_socket.sendto(response.encode(), client_address)

print(f'getsockname: {server_socket.getsockname()}')
print(f'gethostname: {socket.gethostname()}')
print(f'gethostbyname:
{socket.gethostbyname(socket.gethostname())}')
print(f'gethostbyname_ex:
{socket.gethostbyname_ex(socket.gethostname())}')
print(f'gethostbyaddr:
{socket.gethostbyaddr(socket.gethostname())}')

# 关闭服务器的监听套接字（在实际应用中通常不会在这里关闭，而是让服务器持续运行）
server_socket.close()
```

Client:

```
import socket

# 创建一个 IPv4 的 UDP 套接字
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# 定义服务器的地址和端口
server_address = ('127.0.0.1', 43999)

# 客户端向服务器发送经过编码的消息“我是客户端”
message = '我是客户端'.encode()
client_socket.sendto(message, server_address)

# 客户端接收来自服务器的最多 1024 字节的数据以及服务器的地址
data, server_addr = client_socket.recvfrom(1024)
# 打印接收到的来自服务器的响应消息
print(f'来自服务器的响应为：{data.decode()}')

# 关闭客户端套接字
client_socket.close()
```

华清远见元宇宙实验中心 yyzlab.com.cn

4.4 循环服务器

华清远见 HQYJ.COM

4.4.1 TCP服务器

Server：

```
import socket


# 1. 创建套接字
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2. bind 绑定，用来将网络服务绑定到指定的IP地址和端口号
server_address = ('127.0.0.1', 43999)
server_socket.bind(server_address)

# 3. listen
server_socket.listen(5)

print('服务器已启动成功...')


# 4. accept
while True:
    client_socket, client_address = server_socket.accept()
    print(f'{client_address}已连接...')
    while True:
        # 5. recv
        data = client_socket.recv(1024)
        if not data:
            print(f'{client_address}客户端已断开连接..')
            break
        data_decode = data.decode()
        print(f'接收到了来自{client_address}的消息：{data_decode}')

        # 6. send
        if data_decode == 'the last one':
            message = 'ok'.encode()
        else:
            message = '服务器收到了消息'.encode()
        client_socket.send(message)
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
# 7. 在没有任何消息传来的时候或不需要服务器的时候，就可以使用close去关闭服务器
server_socket.close()
```

Client:

```
import socket
import time

# 创建一个 IPv4 的 TCP 套接字
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 定义服务器的地址和端口
server_address = ('127.0.0.1', 43999)

# 客户端连接到服务器的地址和端口
client_socket.connect(server_address)

print('已成功连接服务器...')
i = 0

while True:
    if i == 3:
        # 客户端向服务器发送经过编码的消息“我是客户端”
        client_socket.send('the last one'.encode())
    else:
        client_socket.send(str(i).encode())
    # 客户端接收来自服务器的最多 1024 字节的数据
    data = client_socket.recv(1024)
    # 打印接收到的来自服务器的响应消息
    print(f'来自服务器的响应为：{data.decode()}')
    if data.decode() == 'ok':
        break
    i += 1
    time.sleep(1)

# 关闭客户端套接字
client_socket.close()
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

4.4.2 UDP服务器

Server:

```
import socket

# 创建一个 IPv4 的 UDP 套接字
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# 定义服务器的地址和端口
server_address = ('127.0.0.1', 43999)

# 将套接字绑定到指定的地址和端口
server_socket.bind(server_address)

print('服务器正在监听...')

while True:
    # 接收客户端发送的数据以及客户端的地址
    data, client_address = server_socket.recvfrom(1024)
    print(f'接收来自{client_address}的数据: {data.decode()}')
    if data.decode() == 'the last one':
        server_socket.sendto('ok'.encode(), client_address)
        print(f'客户端{client_address}准备断开连接...')
        # break
    else:
        # 定义服务器要发送给客户端的响应内容
        response = '我是服务器'
        # 将响应内容发送给客户端
        server_socket.sendto(response.encode(), client_address)

# 关闭服务器的监听套接字
server_socket.close()
```

Client:

```
import socket
import time

# 创建一个 IPv4 的 UDP 套接字
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

华清远见云课实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
# 定义服务器的地址和端口
server_address = ('127.0.0.1', 43999)

i = 0

while True:
    if i == 3:
        # 客户端向服务器发送经过编码的消息“我是客户端”
        message = 'the last one'.encode()
        client_socket.sendto(message, server_address)
    else:
        message = str(i).encode()
        client_socket.sendto(message, server_address)
    # 客户端接收来自服务器的最多 1024 字节的数据以及服务器的地址
    data, server_addr = client_socket.recvfrom(1024)
    # 打印接收到的来自服务器的响应消息
    print(f'来自服务器的响应为：{data.decode()}')
    if data.decode() == 'ok':
        break
    i += 1
    time.sleep(1)

# 关闭客户端套接字
client_socket.close()
```

4.5 并发服务器

4.5.1 多进程

Server：

```
import socket
from multiprocessing import Process


def func(connection, client_address):
    print(f'{client_address}已连接')
    while True:
        try:
            # 从连接套接字接收最多 1024 字节的数据
            data = connection.recv(1024)
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
            # 打印接收到的来自客户端的数据
            print(f'接收来自{client_address}的数据：{data.decode()}')
            response = '你发过来的数据是' + data.decode()
            connection.send(response.encode())

        except ConnectionAbortedError:
            print(f'{client_address}连接已断开')
            # 关闭与客户端的连接套接字
            connection.close()
            break


if __name__ == '__main__':
    # 创建一个 IPv4 的 TCP 套接字
    server_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)

    # 定义服务器的地址和端口
    server_address = ('127.0.0.1', 43999)

    # 将套接字绑定到指定的地址和端口
    server_socket.bind(server_address)

    # 开始监听连接请求，参数 5 表示允许排队等待的连接数量
    server_socket.listen(5)

    print('服务器已启动...')

    while True:
        # 接受一个客户端的连接请求，返回一个新的连接套接字和客户端地址
        connection, client_address = server_socket.accept()
        Process(target=func, args=(connection,
client_address)).start()
```

Client:

```
import socket
import time

# 创建一个 IPv4 的 TCP 套接字
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
# 定义服务器的地址和端口
server_address = ('127.0.0.1', 43999)

# 客户端连接到服务器的地址和端口
client_socket.connect(server_address)

i = 0

while True:
    if i == 3:
        break
    client_socket.send(str(i).encode())
    # 客户端接收来自服务器的最多 1024 字节的数据
    data = client_socket.recv(1024)
    # 打印接收到的来自服务器的响应消息
    print(f'来自服务器的响应为：{data.decode()}')
    i += 1
    time.sleep(1)

# 关闭客户端套接字
client_socket.close()
```

4.5.2 多线程

S:

```
import socket
from threading import Thread


def func(connection, client_address):
    print(f'{client_address}已连接')
    while True:
        try:
            # 从连接套接字接收最多 1024 字节的数据
            data = connection.recv(1024)
            # 打印接收到的来自客户端的数据
            print(f'接收来自{client_address}的数据：{data.decode()}')
            response = '你发过来的数据是' + data.decode()
            connection.send(response.encode())

        except ConnectionAbortedError:
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
            print(f'{client_address}连接已断开')
            # 关闭与客户端的连接套接字
            connection.close()
            break


if __name__ == '__main__':
    # 创建一个 IPv4 的 TCP 套接字
    server_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)

    # 定义服务器的地址和端口
    server_address = ('127.0.0.1', 43999)

    # 将套接字绑定到指定的地址和端口
    server_socket.bind(server_address)

    # 开始监听连接请求，参数 5 表示允许排队等待的连接数量
    server_socket.listen(5)

    print('服务器已启动...')

    while True:
        # 接受一个客户端的连接请求，返回一个新的连接套接字和客户端地址
        connection, client_address = server_socket.accept()
        Thread(target=func, args=(connection,
client_address)).start()
```

C:

```
import socket
import time

# 创建一个 IPv4 的 TCP 套接字
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 定义服务器的地址和端口
server_address = ('127.0.0.1', 43999)

# 客户端连接到服务器的地址和端口
client_socket.connect(server_address)
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
i = 0

while True:
    if i == 3:
        break
    client_socket.send(str(i).encode())
    # 客户端接收来自服务器的最多 1024 字节的数据
    data = client_socket.recv(1024)
    # 打印接收到的来自服务器的响应消息
    print(f'来自服务器的响应为：{data.decode()}')
    i += 1
    time.sleep(1)

# 关闭客户端套接字
client_socket.close()
```

4.5.3 协程

在`asyncio`库中有关于网络编程的一些函数，它们处于事件循环下面：

1. loop.sock_recv(sock, nbytes)：异步版本的socket.recv()，从sock接收至多nbytes大小的消息。
2. loop.sock_sendall(sock, data)：异步版本的socket.sendall()，向sock发送data，该方法会持续发送直到数据发送完毕或抛出异常。
3. loop.sock_connect(sock, address)：异步版本的socket.connect()，用来连接服务器。
4. loop.sock_accept(sock)：异步版本的socket.accept()，用来让服务器接收连接。

> 这些方法要求socket必须是非阻塞型的，所以需要设置socket.setblocking

```
import asyncio
import socket

async def handle_client(client_socket, client_address, loop):
    print(f'{client_address}已成功连接服务器...')
    try:
        while True:
            data = await loop.sock_recv(client_socket, 1024)
            if not data:
                print(f'{client_address}已断开连接...')
                break
            print(f"{client_address}发来消息：{data.decode()}")
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
            message = '你发过来的消息是：' + data.decode()
            await loop.sock_sendall(client_socket,
message.encode())
    except Exception as e:
        print(f'{client_address}连接出现问题：{e}')
    finally:
        client_socket.close()

async def main():
    server_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    server_socket.bind(('127.0.0.1', 43999))
    server_socket.listen(5)
    server_socket.setblocking(False)
    print('服务器已启动...')

    loop = asyncio.get_running_loop()
    while True:
        client_socket, client_address = await
loop.sock_accept(server_socket)
        asyncio.create_task(handle_client(client_socket,
client_address, loop))


asyncio.run(main())
```

```
import socket
import asyncio


async def main():
    client_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    server_address = ('127.0.0.1', 43999)
    client_socket.connect(server_address)
    client_socket.setblocking(False)
    print('已成功连接服务器...')

    loop = asyncio.get_running_loop()
    while True:
        try:
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
            await loop.sock_sendall(client_socket, '我是客户端'.encode())
            data = await loop.sock_recv(client_socket, 1024)
            print('从服务器返回的数据是：', data.decode())
            await asyncio.sleep(1)
        except Exception as e:
            print('客户端异常，连接已退出...')
            client_socket.close()
            break

asyncio.run(main())
```

除了底层API之外，协程里还封装了一些高级API：

1. **asyncio.start_server**：用于服务器端，用来创建并启动服务器并对连接到的客户端进行绑定。

```
asyncio.start_server(client_connected_cb,
                     host=None,
                     port=None,
                     *,
                     loop=None,
)
```

- **client_connected_cb**: 必需的回调函数，当有新的客户端连接到服务器时，会自动调用这个回调函数来处理与该客户端的通信。这个函数应该是一个协程函数，它接受两个参数：`reader` 和 `writer`。`reader` 是一个实现了异步读取数据的方法的对象，`writer` 是一个实现了异步写入数据的方法的对象。
- **host**: 可选参数，用于指定服务器的IP地址或域名。如果为None，则默认监听所有可用接口（IPv4或IPv6）。
- **port**: 可选参数，用于指定服务器的端口号。如果为None，则默认随机选择一个可用端口。
- **loop**: 可选参数，用于指定一个事件循环。如果未提供，则使用默认的事件循环，默认为None即可。

该函数返回的是一个`asyncio.Server`对象。这个对象代表了正在运行的服务器，可以用于停止服务器，也可以通过其属性获取一些关于服务器的信息，比如`server.sockets`可以获取服务器正在监听的套接字列表等。

2. **asyncio.open_connection**：用于客户端，用于建立到指定主机和端口的异步连接，并返回一对`(reader, writer)`对象，其中`reader`用于读取来自连接

华清远见|元宇宙实验中心 yyzsb.com.cn

的数据，`writer` 用于向连接写入数据。

```
asyncio.open_connection(host=None,
                        port=None,
                        *,
                        loop=None,
)
```

- **host**：指定要连接的服务器地址。如果为 `None`，会根据其他参数（如 `sock`）的情况进行处理。如果没有提供套接字并且 `host` 也为 `None`，则会引发异常。
- **port**：指定要连接的服务器端口号。如果为 `None`，并且没有提供套接字，同样会引发异常。
- **loop**: 可选参数，用于指定一个事件循环。如果未提供，则使用默认的事件循环，默认为None即可。

3. **reader.read(n=-1)**：至多读取 *n* 个byte。 如果没有设置 *n* , 则自动置为 `-1` ，-1时表示读至 EOF 并返回所有读取的byte。
4. **writer.write(data)**：此方法会尝试立即将 *data* 写入到下层的套接字。 如果写入失败，数据会被排入内部写缓冲队列直到可以被发送。此方法应当与 `drain()` 方法一起使用。
5. **writer.drain()**：阻塞当前协程，直到底层的缓冲区有足够的空间可以继续写入数据，或者数据已经全部发送出去，从而确保数据能够正确、完整地发送到对端。
6. **writer.wait_closed()**：等待直到流被关闭。应当在close()之后被调用以便等待直到下层的连接被关闭。

Server：

```
import asyncio


# 定义一个异步函数，用于处理每个客户端连接
async def handle_client(reader, writer):
    addr = writer.get_extra_info('peername')  # 获取客户端的地址信息
    print(f"{addr} 已连接")  # 打印客户端连接的信息

    try:
        # 使用一个无限循环来持续读取客户端数据
        while True:
            data = await reader.read(1024)  # 非阻塞地读取最多1024字
节的数据
```

华清远见 HQYJ.COM
华清远见云实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
            # 检查是否读取到空字节串，如果是，则客户端已断开连接
            if not data:
                print(f"{addr} 连接已断开")
                break
            # 如果有数据，则打印接收到的数据
            print(f"接收来自{addr}的数据：{data.decode()}")
            # 构造响应消息，并发送回客户端
            response = '你发过来的数据是' + data.decode()
            writer.write(response.encode())
            await writer.drain()  # 确保所有写入的数据都被发送出去
    except Exception as e:
        # 打印其他可能的异常
        print(f"处理{addr}时发生异常：{e}")
    finally:
        # 确保关闭连接
        writer.close()
        await writer.wait_closed()  # 等待连接真正关闭


# 主函数，用于启动服务器
async def main():
    # 创建服务器，指定处理客户端连接的回调函数，以及服务器监听的地址和端口
    server = await asyncio.start_server(handle_client,
'127.0.0.1', 43999)

    # 获取服务器监听的地址信息
    addr = server.sockets[0].getsockname()
    print(f'服务器在 {addr} 启动...')  # 打印服务器启动的信息

    async with server:
        # 使用异步上下文管理器来确保服务器正确关闭
        await server.serve_forever()  # 使服务器一直运行，直到显式停止


# 当作为主模块运行时，启动服务器
if __name__ == '__main__':
    asyncio.run(main())  # 使用asyncio.run启动主协程
```

Client:

```
import asyncio
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```python
async def client():
    reader, writer = await asyncio.open_connection('127.0.0.1',
43999)
    addr = writer.get_extra_info('sockname')
    for i in range(3):
        writer.write(str(addr).encode())
        await writer.drain()

        data = await reader.read(1024)
        print(f'来自服务器的响应为：{data.decode()}')
        await asyncio.sleep(1)

    writer.close()
    await writer.wait_closed()


async def main():
    await asyncio.gather(
        client(),
        client(),
        client(),
        # 可以添加更多 client() 调用来模拟多个客户端连接
    )

if __name__ == '__main__':
    asyncio.run(main())
```

4.5.4 IO多路复用

IO 多路复用是一种可以同时监控多个 IO 源（比如多个网络连接、多个文件描述符等）的技术，当其中任何一个 IO 源准备好进行读写操作时，程序就可以进行相应的处理，而不需要阻塞地等待每个 IO 源逐个完成操作。

常见的IO多路复用机制有三种：

1. `select`（Windows、Linux都有）：它是最早出现的 IO 多路复用机制。它通过一个文件描述符集合来监控多个 IO 源，当有 IO 源准备好时，`select`函数返回，然后程序可以遍历集合找出准备好的 IO 源进行处理。但是`select`有一些限制，比如监控的文件描述符数量有限制，并且每次调用都需要重新设置监控的文件描述符集合。

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

2. `poll`（只存在于Linux系统中）：和`select`类似，但没有文件描述符数量的限制。不过，当文件描述符数量很多时，它的性能也不是很高。
3. `epoll`（只存在于Linux 系统中）：`epoll`是一种高效的 IO 多路复用机制。它通过事件驱动的方式，当 IO 源准备好时，内核会将事件通知程序，避免了像`select`和`poll`那样需要不断地遍历文件描述符集合。

而在Python中，select、poll、epoll都在`select`模块中，因此可以通过调用`select`模块来使用IO多路复用完成并发服务器。

```
import socket
import select


server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
address = ('127.0.0.1', 43999)
server_socket.bind(address)
server_socket.listen(5)

print('服务器已启动...')

inputs = [server_socket]

while True:
    rlist, wlist, xlist = select.select(inputs, [], [])
    for sock in rlist:
        if sock is server_socket:
            connection, client_address = sock.accept()
            inputs.append(connection)
        else:
            try:
                data = sock.recv(1024)
                if not data:
                    print(f'{sock.getpeername()}已断开连接')
                    inputs.remove(sock)
                    sock.close()
                else:
                    print(f'接收来自{sock.getpeername()}的数据:
{data.decode()}')
                    response = '你发过来的数据是' + data.decode()
                    sock.send(response.encode())
            except Exception as e:
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
                print(e)
                inputs.remove(sock)
                sock.close()
```

```
import socket
import time
import threading


def func(id):
    # 创建一个 IPv4 的 TCP 套接字
    client_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)

    # 定义服务器的地址和端口
    server_address = ('127.0.0.1', 43999)

    # 客户端连接到服务器的地址和端口
    client_socket.connect(server_address)

    message = str(client_socket.getpeername()) + str(id)
    while True:
        client_socket.send(message.encode())
        # 客户端接收来自服务器的最多 1024 字节的数据
        data = client_socket.recv(1024)
        # 打印接收到的来自服务器的响应消息
        print(f'来自服务器的响应为：{data.decode()}')
        time.sleep(1)


thread_list = []
while True:
    for i in range(200):
        t = threading.Thread(target=func, args=(i, ))
        t.start()
        thread_list.append(t)
    for th in thread_list:
        th.join()
```

使用selectors库：

selectors.DefaultSelector是一个指向当前平台上可用的最高效实现的别名。

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```python
import socket
import selectors

sel = selectors.DefaultSelector()

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
address = ('127.0.0.1', 43999)
server_socket.bind(address)
server_socket.listen(5)

print('服务器已启动...')

sel.register(server_socket, selectors.EVENT_READ, data=None)

while True:
    events = sel.select()
    for key, mask in events:
        if key.fileobj is server_socket:
            connection, client_address = server_socket.accept()
            print(f'接受来自 {client_address} 的连接')
            connection.setblocking(False)
            sel.register(connection, selectors.EVENT_READ,
data=client_address)
        else:
            try:
                sock = key.fileobj
                data = sock.recv(1024)
                if not data:
                    print(f'{sock.getpeername()}已断开连接')
                    sel.unregister(sock)
                    sock.close()
                else:
                    print(f'接收来自{sock.getpeername()}的数据:
{data.decode()}')
                    response = '你发过来的数据是' + data.decode()
                    sock.send(response.encode())
            except Exception as e:
                print(e)
                sel.unregister(sock)
                sock.close()
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

5. 常见的高级网络编程库

1. requests：主要用于发送 HTTP 请求，功能强大且使用方便，可以轻松地发送各种 HTTP 请求，如 GET、POST、PUT、DELETE 等。
2. aiohttp：用于异步网络编程，特别适合高并发场景，基于 asyncio 框架，允许同时处理大量的并发连接，提高程序的性能和响应速度。
3. websockets：专注于 WebSocket 通信，提供了简洁的 API 来建立 WebSocket 连接，发送和接收消息，与 asyncio 集成良好，适用于异步环境下的实时通信。
4. tornado：一个强大的异步网络框架，采用非阻塞的 I/O 模型和异步编程，能够处理大量的并发连接，在早期版本中与`asyncio`不兼容，但在较新版本中可以与`asyncio`一起使用。

华清远见|元宇宙实验中心
yyzlab.com.cn